ONBRD-03: Deploying ActiveGate

> **Series:** ONBRD — Dynatrace Onboarding | **Notebook:** 3 of 10 | **Created:** December 2025 | **Last Updated:** 10/22/2026

Your Network Gateway to Dynatrace

ActiveGate is a lightweight component that routes traffic between your infrastructure and Dynatrace. This notebook covers when you need ActiveGate, how many to deploy, where to place them, and installation steps - including comprehensive Kubernetes deployment options.

Table of Contents

1. What is ActiveGate?
2. When Do You Need ActiveGate?
3. How Many ActiveGates?
4. Where to Deploy?
5. Generating Tokens
6. Installation Methods
7. Kubernetes Deployment (Detailed)
8. Verifying Deployment
9. Troubleshooting
10. Next Steps

Prerequisites

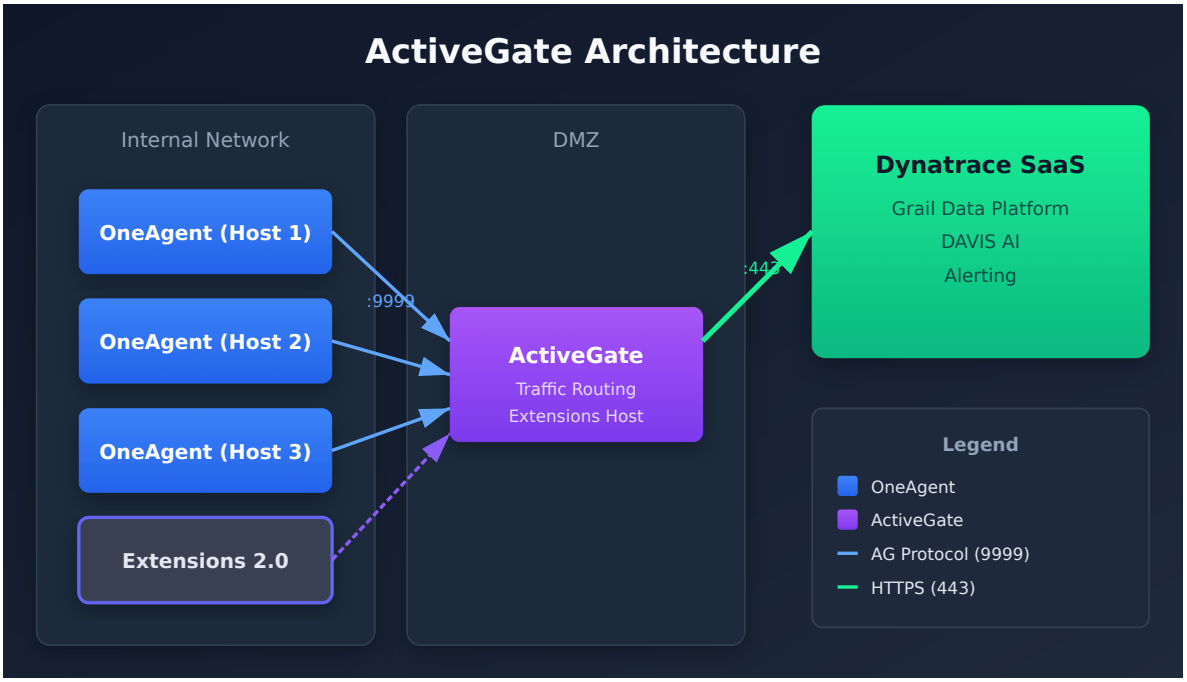
- Admin access to Dynatrace environment
- Network architecture diagram (know your zones)
- Server for ActiveGate (Linux, Windows, or Kubernetes cluster)
- Outbound HTTPS (443) to Dynatrace SaaS

1. What is ActiveGate?

ActiveGate is a proxy and routing component that connects your environment to Dynatrace.

Component	Purpose
Environment ActiveGate	Routes OneAgent traffic and runs extensions; private synthetic monitors run on a *separate* Synthetic-enabled ActiveGate
Cluster ActiveGate	Dynatrace Managed only — the cluster's mediating proxy for OneAgent and digital experience traffic from outside its network

> **Note:** This notebook covers Environment ActiveGate for Dynatrace SaaS. On Managed, *"use one or more Cluster ActiveGates as mediating proxies for pre-processing of OneAgent and DEM traffic"*; both ActiveGate types on Managed are covered in DTM-04: ActiveGate Deployment on Managed. (Managed deployments (DT docs))



ActiveGate Capabilities

Capability	Description
Traffic Routing	Proxies OneAgent data to Dynatrace
Extensions 2.0	Runs Python-based extensions locally
Synthetic Monitoring	Private synthetic locations — on a dedicated Synthetic-enabled ActiveGate only (see the note below)
Cloud Integrations	Classic AWS / Azure metric polling (or use the Clouds app for direct connections); the GCP integration needs no ActiveGate on SaaS
Kubernetes API	Cluster monitoring via API
Log Ingest	Generic log ingest endpoint

> **Synthetic ActiveGates do nothing else.** *"A clean ActiveGate installation for the purpose of synthetic monitoring disables all other ActiveGate features, including communication with OneAgents."* Plan private synthetic locations

> on their own ActiveGates, installed from the **Install ActiveGate** wizard in **Discovery & Coverage** with **Purpose → Synthetic**, and never count them toward routing capacity. (Install a Synthetic-enabled ActiveGate (DT docs))
>
> **OneAgent Attribute Enrichment (OneAgent 1.333+):** OneAgent can enrich all telemetry (metrics, spans, logs, events) with primary fields (`dt.security_context` , `dt.cost.costcenter`) and primary tags (`primary_tags.environment` , `primary_tags.team`) at the source. More efficient than auto-tags — feeds directly into OpenPipeline routing, bucket assignment, and Grail permissions. Configure via the installer's `--set-host-tag` at install time, or `oneagentctl --set-host-tag … --restart-service` afterwards (set parameters apply only after a OneAgent restart). See docs.

Dynatrace Version Support Policy

Component	Standard Support	Enterprise Support
OneAgent	9 months	12 months
ActiveGate	9 months	12 months
Dynatrace Operator	Independent release cycle — check release notes	

> **Version floor (1.347 release notes).** The OneAgent 1.347 and ActiveGate 1.347 release notes (published 10/01/2026; staged rollout from 09/30/2026) list the oldest supported version as **1.329** (Standard Support) and **1.323** (Enterprise Success and Support) — the floor moves with every release, so re-read it from the current release notes rather than from this table. Separately, **SaaS 1.347** (staged tenant rollout) *rejects* connections from **OneAgent 1.241 and earlier** once it reaches your tenant — those hosts stop reporting rather than merely falling out of support. See **FAQ-04: Managing OneAgent updates on Dynatrace SaaS**. Sources: OneAgent 1.347 (DT docs), ActiveGate 1.347 (DT docs), SaaS 1.347 (DT docs) — *"Starting with this release, Dynatrace rejects connections from OneAgent versions 1.241 and earlier."*

Technology Support Levels

Dynatrace defines two support levels for third-party technologies:

Level	Meaning (quoted)
Supported	*"We provide support for any problems directly caused by Dynatrace."*
Limited support	*"Dynatrace provides support for a limited set of functionality for a particular technology."*

Third-Party Technology EOL Policy

"Dynatrace typically supports technologies and their versions six months longer than the vendor to give you enough time to upgrade your environment." "End of support announcements are provided six months in advance." Treat the six months as the usual pattern, not a guarantee — read the announcement for the technology you run.

> **Reference:** Technology support (DT docs) | End-of-support announcements (DT docs) | Support Policy

2. When Do You Need ActiveGate?

Required Scenarios

Scenario	Why ActiveGate is Required
Network-restricted hosts	OneAgents can't reach internet directly
Private synthetic monitors	Test internal applications
Remote extensions	Extensions that poll remote sources (SNMP, databases, APIs) run from an ActiveGate group; local extensions run on OneAgent
AWS / Azure classic integration	See Clouds app status below — classic polling runs through an ActiveGate (on SaaS hosted on AWS, Dynatrace provides one for the built-in AWS services); GCP needs none on SaaS
VMware monitoring	vCenter integration
Kubernetes full-stack	Cluster API access for events, metrics

Update — Clouds App (status as of 10/2026): The **Clouds app** supports direct cloud connections **without ActiveGate** — but coverage varies per cloud:

Cloud	Clouds-app Status	AG Required?
AWS	GA — direct connection supported	No (when using Clouds app)
Azure	Direct connection (SaaS 1.337+) — *"no need to deploy ActiveGate compute resources for metric polling"*	No (when using Clouds app)
GCP	Clouds connection listed as **Preview** on the setup page; SaaS 1.348 (pre-release, rollout planned from 09/22/2026) announces it with *"no need to deploy ActiveGate compute resources for metric polling within your Google Cloud environment"* — verify it has reached your tenant	**No.** The classic integration (`dynatrace-gcp-monitor` , deployed on GKE or as a Cloud Function) sends straight to your environment URL on SaaS and remains the working path until the Clouds connection reaches you

ActiveGate is still required for **remote extensions**, **private synthetic locations**, and classic AWS / Azure polling where the Clouds app is not used. On SaaS, the GCP collector's *"dynatraceUrl"* is *"your environment URL"*; an ActiveGate appears there only as a Managed log-ingest option. See Clouds app documentation, Azure Cloud Platform Monitoring (DT docs), Set up Dynatrace on Google Cloud (DT docs) and the **CLOUD series** for per-provider deep dives.

Sources: Set up the Dynatrace GCP integration on GKE (DT docs) — *"For Managed deployments: You can use an existing ActiveGate for log ingestion."*, SaaS 1.348 (DT docs), AWS CloudWatch metrics (DT docs) — *"An ActiveGate capable of monitoring your AWS account for classic (built-in) supported services is already provided and available within the Dynatrace AWS account (only for SaaS environments hosted on AWS)."*, Extensions (DT docs) — *"Run extensions locally on the monitored host to collect data from local data sources with the Extension Execution Controller."*

Routing OneAgent Traffic: Through an ActiveGate or Direct?

When every host can already reach your SaaS environment, routing OneAgent traffic through an ActiveGate is a design choice, not a requirement. *"In the absence of an ActiveGate, data collected by OneAgent is sent directly to the Dynatrace Cluster"*, and the OneAgent install guide puts it the same way: you *"may choose to either provide direct access to the Dynatrace cluster or set up an ActiveGate."* Each side has costs:

Factor	Route through an ActiveGate	Go direct to your SaaS environment
Egress and allowlisting	Only the ActiveGates need outbound 443. For hosts without internet access, Dynatrace says to *"install an ActiveGate to serve as a single access point, rather than opening the firewall for multiple hosts running OneAgents"*, so the firewall and proxy rules name a few ActiveGates.	Every monitored host needs outbound HTTPS 443 to your environment, through your firewall or proxy.
Proxy configuration	Configured on the ActiveGates: *"You can use a single proxy for all outgoing traffic"*, set in the ActiveGate configuration. Dynatrace says the single-access-point approach *"greatly reduces the effort of managing and maintaining firewall and/or proxy configuration settings."*	Each OneAgent host that reaches the environment through a proxy needs that proxy set on the host, with `oneagentctl --set-proxy` or at install time.
Network overhead	The routing module *"collects messages from OneAgent instances and builds bulks, which are then sent in compressed form"*. Dynatrace says this *"can reduce network overhead considerably, depending on the number of OneAgent instances"* behind the ActiveGate.	No ActiveGate-side bulking: each OneAgent connects to the environment on its own.
Connections at the egress	OneAgent connections end at the ActiveGates, and only the ActiveGates hold connections out through your NAT or firewall. Dynatrace says the ActiveGate lets you *"reduce your interaction with Dynatrace to one single point—available locally"*, and that it *"reduces the complexity of network connections."*	Every OneAgent on every host holds its own connections through the shared egress device. The docs publish no per-host connection count. In field experience it grows with the number of instrumented processes on the host (see the note below).
Infrastructure to run	ActiveGate hosts to size (§ 3 and FAQ-10), keep updated (FAQ-05) and run at two or more per network zone for high availability. Auto-update covers host-based ActiveGates only: *"Auto-update and One-click update functionalities are limited to host-based—using installer—deployment only."*	Nothing to deploy, size, patch or update.
Capacity	The ActiveGates in a zone carry every host behind them, so their capacity, and the survivor's capacity when one is down (§ 3), caps the zone.	No intermediate tier to saturate.
Operating-system lifecycle	Every ActiveGate host must stay on a supported OS. The § 3 table lists nine OS versions that lose ActiveGate support between 11/01/2026 and 01/01/2027.	No additional OS to track.
Locality and failover	Network zones keep OneAgents on nearby ActiveGates, and *"You can still use ActiveGates in other zones as a backup option in case primary ActiveGates becomes unavailable"*. See the fallback table below.	No zone design. Resilience is the host's own path to your environment.
Installer cache	The requirements page budgets 4 GB for *"ActiveGate and OneAgent cached installers and container images—if such will need to be stored."*	Hosts download installers from the environment.

Factor	Route through an ActiveGate	Go direct to your SaaS environment
Reuse	A routing/monitoring ActiveGate you already run for remote extensions, Kubernetes monitoring or classic cloud polling can also route OneAgent traffic. A Synthetic-enabled ActiveGate cannot: a clean Synthetic installation *"disables all other ActiveGate features, including communication with OneAgents"* (§ 1).	No reuse needed.
Memory dumps	OneAgent uploads Java, .NET and Node.js memory dumps to the ActiveGate, which stores them for download. Dynatrace *"doesn't allow you to upload dumps to the Dynatrace cluster in the cloud."*	No central store. Dumps stay on the monitored host only for a limited time: *"when OneAgent periodically empties the directory, the file size could be 0 bytes."* Copy a dump off the host before it ages out.
z/OS	The zRemote module *"routes that data, compressed and encrypted, via its local ActiveGate to Dynatrace."*	Not possible: z/OS monitoring always goes through an ActiveGate.

Recommended approach: route through ActiveGates when hosts cannot or should not have direct egress, when your security team wants monitoring traffic to leave through a few known hosts, or when a large fleet shares one NAT or firewall egress (see the note below). Go direct when hosts already have outbound access, the fleet behind each egress is small, and nothing else requires an ActiveGate (central memory dump storage and z/OS monitoring both do). Choose per network zone: a sealed data center can route through ActiveGates while cloud workloads with egress go direct.

> **Large fleets going direct can exhaust the egress device.** In field experience, each instrumented process on a full-stack host talks to your environment on its own. A host with log monitoring and 20 injected processes then holds on the order of 25 TCP connections. Dynatrace does not publish that figure, so measure it on one of your own hosts before planning around it: on Linux, `ss -tn state established '( dport = :443 )'` , then count the lines whose peer is your environment's address.
>
> Across a fleet, all of those connections pass through the NAT or firewall the hosts share for internet egress. A device that translates everything to **one public IP**, from a single port pool shared across destinations, has at most 65,535 source ports, and fewer once reserved ranges are excluded. 2,000 hosts × 25 connections uses about 50,000 of them and leaves roughly 15,000 for everything else on the network. At 30 instrumented processes per host the pool is gone: some OneAgent connections fail, and so does other outbound traffic. Devices with a pool of public IPs, or that allocate ports per destination, have more port headroom, but their session table still has to hold every connection.
>
> One large enterprise deployment that sent its whole fleet direct saw sites lose internet connectivity this way until it deployed ActiveGates. Before going direct at scale, ask your network team for two numbers: the egress device's NAT port capacity and its session-table limit.
>
> **Kubernetes clusters usually run an ActiveGate anyway.** Kubernetes/OpenShift monitoring is an ActiveGate module, and the Operator's full-stack example deploys an ActiveGate with the `routing` and `kubernetes-monitoring` capabilities. In-cluster OneAgents can still go direct, so for a cluster the question is usually whether to route through the ActiveGate you already run there (§ 7), not whether to deploy one.
>
> **Going direct does not stay direct by itself.** The connectivity scheme says *"OneAgent will connect to an Environment ActiveGate, if one is present"*, and OneAgents with no network zone *"try to connect to any ActiveGate in a random order"*. If you later deploy an Environment ActiveGate for extensions or Kubernetes, hosts that can reach it may start routing through it. Use network zones to decide which hosts use which ActiveGates.

When every ActiveGate in a zone is down

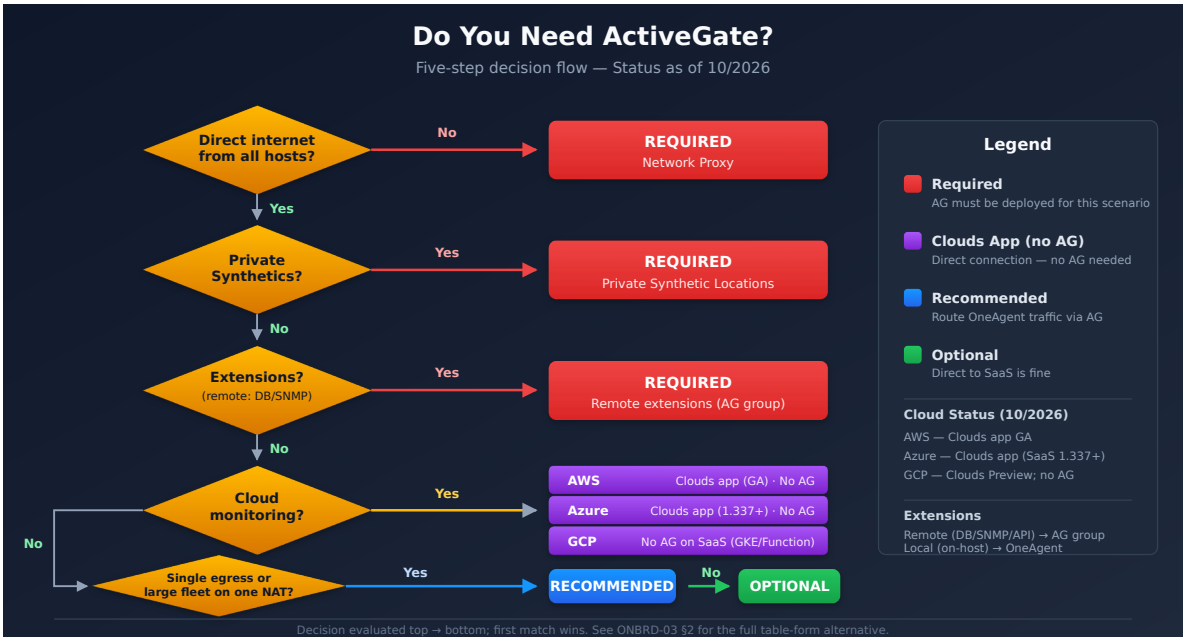
Each network zone has a **fallback mode** (**Deployment Status → Network zones → Edit → Fallback mode**). It decides what OneAgents do when *"no ActiveGates are available in this network zone and there are no alternative network zones"*:

Fallback mode	Behavior when no ActiveGate in the zone or its alternative zones is available
Any ActiveGate (default)	*"traffic is routed to any available ActiveGate, including an ActiveGate in the default zone."*
Only default zone	*"traffic is routed only to an available ActiveGate in the default zone. ActiveGates in other zones aren't considered."*
None	*"Traffic stays within the primary zone and its alternative zones. If no ActiveGate in those zones is available, OneAgents don't send data to another zone."*

On SaaS, the connectivity scheme says OneAgent connects to the SaaS cluster *"directly, if no connection to an Environment ActiveGate is possible."* Fallback only helps if the host can reach the fallback target. A host in a sealed network that loses every ActiveGate it can reach has nowhere to send data, so plan two or more ActiveGates per zone (§ 3) and configure alternative zones the hosts can reach.

> **Sources:**
> - Data security controls (DT docs), Install OneAgent on Linux (DT docs)
> - Route OneAgent traffic to Dynatrace (DT docs) — *"This can reduce network overhead considerably, depending on the number of OneAgent instances communicating with the ActiveGate and on the amount of data transferred by them."* and *"In this way Dynatrace ActiveGate allows you to reduce your interaction with Dynatrace to one single point—available locally."*
> - Supported connectivity schemes for ActiveGates (DT docs) — *"Dynatrace SaaS Cluster receives connections on port 443."*
> - Get started with network zones (DT docs), OneAgent connectivity in network zones (DT docs) — *"None (drop traffic) corresponds to priority groups 1 and 2."*
> - Proxy for ActiveGate (DT docs), OneAgent configuration via command-line interface (DT docs)
> - Install a Synthetic-enabled ActiveGate (DT docs)
> - Analyze memory dumps (DT docs) — *"Memory dumps are stored by OneAgent locally for a limited time on the disk of the monitored application-server machine."*
> - Install the zRemote module for z/OS monitoring (DT docs), ActiveGate purposes and functionality (DT docs), Full-stack observability on Kubernetes (DT docs)
> - Update ActiveGate (DT docs), Linux ActiveGate hardware and system requirements (DT docs), End-of-support announcements (DT docs)
> - **Derived:** the recommended approach and the "does not stay direct" note combine the connectivity-scheme preference for an Environment ActiveGate with the unconfigured-OneAgent behavior on the network-zones page.

Decision Tree



3. How Many ActiveGates?

Sizing Guidelines

> **FAQ-10 owns the ActiveGate sizing decision.** *FAQ-10: How do I size and scale ActiveGates?* carries the current published per-shape capacity tables, the 50% CPU / 80% memory headroom rule, the survivor-capacity HA math for a multi-node zone, and the `dt.sfm.active_gate.*` saturation signals to watch once the fleet is running. Treat the table below as an order-of-magnitude sanity check while you plan the deployment — size the actual machines against FAQ-10 and the official requirements page.

Dynatrace publishes estimated host counts per reference instance shape for an ActiveGate that only routes OneAgent traffic. The counts assume a specific workload: *"On each host, OneAgent is performing eight monitoring tasks"* — infrastructure monitoring, log monitoring, full-stack monitoring of 3 Apache Tomcat and 2 Apache HTTP Server instances, and extension monitoring. The page adds that *"The real number of hosts may be different depending on the monitored technologies in your environment."* In steady state the ActiveGate machine *"should not exceed 50% CPU and 80% memory"*.

Reference shape	Architecture	vCPU	RAM	Estimated hosts routed
c6i.large	x86-64	2	3.75 GiB	~800
c6i.xlarge	x86-64	4	7.5 GiB	~1,800
c6i.2xlarge	x86-64	8	15 GiB	~2,500
c7g.large	ARM64	2	3.75 GiB	~1,300
c7g.xlarge	ARM64	4	7.5 GiB	~2,700
c7g.2xlarge	ARM64	8	15 GiB	~5,500

Reference shape	Architecture	vCPU	RAM	Estimated hosts routed
S	s390	2	4 GiB	~800
M	s390	4	8 GiB	~1,500

The reference points are AWS instance shapes (plus two s390 machine sizes). For on-premises VMs, map by vCPU and RAM rather than looking for the instance name. Compare your own hosts with the eight-task profile before reusing a count: a host with more instrumented processes than that profile fills an ActiveGate sooner.

Check the architecture before you pick an instance family. ARM64 routes more hosts per vCPU than x86-64 at the same shape, but *"ActiveGate installed on the x86-64 architecture supports all functionalities. Other architectures provide only partial support."* Confirm that every ActiveGate purpose you need (extensions, cloud polling, Kubernetes monitoring) works on ARM64 or s390 before you standardize on it. Windows ActiveGates are x86-64 only: *"ActiveGate supports only operating systems running on the x86-64 architecture (64-bit Intel/AMD)."*

Routing plus log ingest on the same ActiveGate

The table above is for routing only. An ActiveGate that also receives logs through the Log ingestion API has its own table, in which *"OneAgent routing traffic is set at 50% of the routing-only capacity for the machine size"*:

Reference shape	Architecture	vCPU	RAM	Estimated hosts routed	API log ingestion (MB/min)
c7i.large	x86-64	2	3.75 GiB	~400	~750
c7i.xlarge	x86-64	4	7.5 GiB	~900	~1,500
c7i.2xlarge	x86-64	8	15 GiB	~1,250	~3,300
c7g.large	ARM64	2	3.75 GiB	~400	~280
c7g.xlarge	ARM64	4	7.5 GiB	~1,350	~560
c7g.2xlarge	ARM64	8	15 GiB	~2,750	~1,650

So a 2-vCPU x86-64 ActiveGate that routes about 800 hosts on its own routes about 400 once it also takes API log ingest. If your ActiveGates will front log ingestion, size them from this table, not the routing-only one. An ActiveGate that receives logs **only** through the API has a third table; FAQ-10 reproduces it.

> **Sources:** Linux ActiveGate hardware and system requirements (DT docs) — *"It is recommended that the machine on which ActiveGate is running should not exceed 50% CPU and 80% memory."*, Windows ActiveGate hardware and system requirements (DT docs) — *"ActiveGate supports only operating systems running on the x86-64 architecture (64-bit Intel/AMD)."* **Derived:** the 800 → 400 comparison maps the c6i and c7i shapes by vCPU and RAM.

Detailed Hardware Requirements

Minimum System Requirements (Linux)

Component	Documented requirement
CPU	1 dual core processor (more for large environments)
RAM	2 GB (4 GB recommended)

Component	Documented requirement
Disk	4 GB for installation, configuration and logs, plus 4 GB for cached OneAgent / ActiveGate installers and container images
Disk, ActiveGates that run extensions	Add 600 MB plus a 1.5 GB buffer for the Extension Execution Controller's log-retransmission persistence file, and space for the extensions you upload
Disk, dump files (optional)	Off by default; when turned on, configurable with a 100 GB default maximum

Don't skip the extension buffer. For the persistence directory the page warns that *"The reliability mechanism does not work if the requirement is not met."* Extensions keep running, but without the buffer that covers an outage. Windows ActiveGates have the same minimums.

> **Sources:** Linux ActiveGate hardware and system requirements (DT docs) — *"2 GB RAM (4 GB recommended). 1 dual core processor."*, *"600 MB + 1.5 GB (buffer) of free disk space for Extension Execution Controller logs retransmission persistence file."*, Windows ActiveGate hardware and system requirements (DT docs).

Operating System Support

The routing/monitoring ActiveGate matrix changes every few sprints, so **the official requirements pages are the authority** — re-check them before you provision, not just when something breaks: Linux ActiveGate hardware and system requirements and Windows ActiveGate hardware and system requirements.

Currently listed for routing/monitoring ActiveGates (verified 10/21/2026 against the Linux page, updated 07/07/2026, and the Windows page, updated 07/15/2026). Architectures vary by distribution: ARM64 and x86-64 throughout, s390 on RHEL, SUSE and Ubuntu 20.04+. Only x86-64 supports every function (see above).

OS	Supported Versions
Red Hat Enterprise Linux	8.10, 9.4, 9.6, 9.7, 9.8, 10.0, 10.1, 10.2
Oracle Linux	8.10, 9.7, 9.8, 10.1, 10.2
Rocky Linux	8.10, 9.7, 9.8, 10.1, 10.2
Ubuntu	16.04, 18.04, 20.04, 22.04, 24.04, 26.04 (x86-64); 20.04, 22.04, 24.04, 26.04 (ARM64 / s390)
Amazon Linux	2, 2023
SUSE Linux Enterprise Server	15.7
Windows Server	2016, 2019, 2022, 2025 (x86-64 only)

Added in ActiveGate 1.343: Oracle Linux 9.8 and 10.2, Rocky Linux 9.8 and 10.2. ActiveGate 1.343 was published 07/15/2026 with a staged fleet rollout from 07/28/2026 — the ActiveGates already running in your environment carry whatever version they last updated to, so check the version of the specific ActiveGate before assuming a newly added OS is installable on it.

> **On CentOS and Debian:** neither CentOS Linux, CentOS Stream, nor Debian appears on the current routing/monitoring ActiveGate matrix (verified 07/30/2026). Earlier revisions of this notebook combined "RHEL/CentOS" into a single row, which reads as an endorsement of CentOS that the requirements page does not make — and CentOS Linux is end-of-life upstream regardless of what Dynatrace supports. If you are

> planning ActiveGates on CentOS Stream or Debian, check the requirements page directly before provisioning rather than inferring support from the RHEL row.

Support Ending — Do Not Provision New ActiveGates on These

Dynatrace de-supports an OS version *"at least 6 months after its end of life (EOL), to give you enough time to upgrade your environment."* The announced removals below all fall inside the next six months:

OS version	ActiveGate support ends
Red Hat Enterprise Linux 9.4	**11/01/2026**
Ubuntu 16.04	**11/01/2026**
Red Hat Enterprise Linux 9.7, 10.1	**12/01/2026**
Oracle Linux 9.7, 10.1	**12/01/2026**
Rocky Linux 9.7, 10.1	**12/01/2026**
Amazon Linux 2	**01/01/2027**

Existing ActiveGates on these versions keep working right up to the date — nothing stops the moment the announcement lands. What ends is Dynatrace's commitment to fix issues and ship new ActiveGate versions for that OS. Two practical consequences for an onboarding plan:

- **Do not provision new ActiveGates on any version in this table.** Amazon Linux 2 and RHEL 9.4 were the obvious defaults a year ago and are now the wrong starting point — pick Amazon Linux 2023, RHEL 9.8/10.2, Oracle Linux 9.8/10.2, or Rocky Linux 9.8/10.2 instead. Oracle Linux and Rocky Linux 9.8 and 10.2 are themselves already announced for de-support from **06/01/2027** (ActiveGate 1.347 release notes: *"The following operating systems will no longer be supported starting 01 June 2027"*), so on those distributions plan the move to the next listed minor version. On Windows, *"The following operating systems will no longer be supported starting 01 August 2027 Windows: Windows Server 2016"* — start new Windows ActiveGates on 2019 or later.
- **Fold the OS upgrade into the next ActiveGate update window** rather than scheduling it as separate work. The update mechanics, auto-update behavior, and how to schedule those windows are covered in *FAQ-05: Managing ActiveGate updates on Dynatrace SaaS*.

> **Reference:** End-of-support announcements (DT docs) — the dates and the 6-month policy statement above are quoted from this page, verified 07/30/2026; the dates also match ActiveGate 1.347 (DT docs), read 10/02/2026.

Disk Space per Directory (Linux defaults)

Most of the space an ActiveGate needs sits under `/var`, not under `/opt/dynatrace`:

Directory (default)	Contents	Space
`/opt/dynatrace`	ActiveGate and autoupdater executables and libraries	600 MB

Directory (default)	Contents	Space
`/opt/dynatrace/remotepluginmodule`	Extensions executables (Environment ActiveGate)	2 GB
`/var/lib/dynatrace`	Configuration	2 MB
`/var/lib/dynatrace/packages`	Auto-update installer downloads	600 MB
`/var/log/dynatrace`	ActiveGate and autoupdater logs	1.2 GB
`/var/tmp/dynatrace/gateway`	Temporary files	4 GB (including 3 GB for cached OneAgent installers and container images)
`/var/lib/dynatrace/remotepluginmodule`	Extensions logs, cache and run-time work area (Environment ActiveGate)	2 GB
`/var/lib/dynatrace/remotepluginmodule/agent/runtime/extensions/persistence`	Extension Execution Controller log-retransmission persistence	Up to 600 MB, plus a 1.5 GB buffer
`/opt/dynatrace/remotepluginmodule/plugin_deployment`	Uploaded extensions (Environment ActiveGate)	Depends on the extensions uploaded
`/var/lib/dynatrace/gateway/dump`	Dump files uploaded by OneAgent (off by default)	Configurable; 100 GB default maximum

Added up, the fixed rows come to about 6.5 GB for a routing-only ActiveGate and about 12.5 GB for an ActiveGate that runs extensions, before uploaded extensions and optional dump files.

> **Partitioning:** if you give ActiveGate dedicated volumes, size `/var/tmp/dynatrace`, `/var/log/dynatrace` and `/var/lib/dynatrace` per the requirements page — those are the directories that hold logs, caches and update packages. A large `/opt/dynatrace` partition alone leaves them on the root volume.

> **Sources:** Linux ActiveGate hardware and system requirements (DT docs) — *"ActiveGate temporary files default: /var/tmp/dynatrace/gateway"*. **Derived:** the 6.5 GB and 12.5 GB totals add the table's fixed rows. Memory and CPU sizing beyond the minimums: FAQ-10.

High Availability

The sizing tables assume that *"ActiveGates may be inoperable during updates, restarts or short communication problems"*, and that *"operating ActiveGates should be able to takeover traffic of the unavailable ActiveGates"*. So a production zone needs **at least 2 ActiveGates**, and each one must be able to carry the whole zone while the other is down:

Deployment	ActiveGates	Basis
Minimum HA	2 per zone	The takeover rule above
Load distribution	2–3 per zone	Community practice
Large scale	N+1 per zone	Community practice: size N to carry the zone, then add one

How OneAgents spread across them. *"OneAgents are load-balanced to use an ActiveGate with the lowest possible index. That is, if several ActiveGates with the low index are available, OneAgents will try to switch between the available ActiveGates on a regular basis."* That is periodic switching, not per-request distribution. In community practice the load on each node is uneven over short windows, so watch per-node CPU and memory (FAQ-10) rather than assuming an even split.

> **Sources:** Linux ActiveGate hardware and system requirements (DT docs) — *"In order to ensure high availability, operating ActiveGates should be able to takeover traffic of the unavailable ActiveGates"*, OneAgent connectivity in network zones (DT docs) — *"OneAgents are load-balanced to use an ActiveGate with the lowest possible index."*

Example Deployment

The example assumes hosts close to the eight-task profile and no log ingest through the ActiveGates.

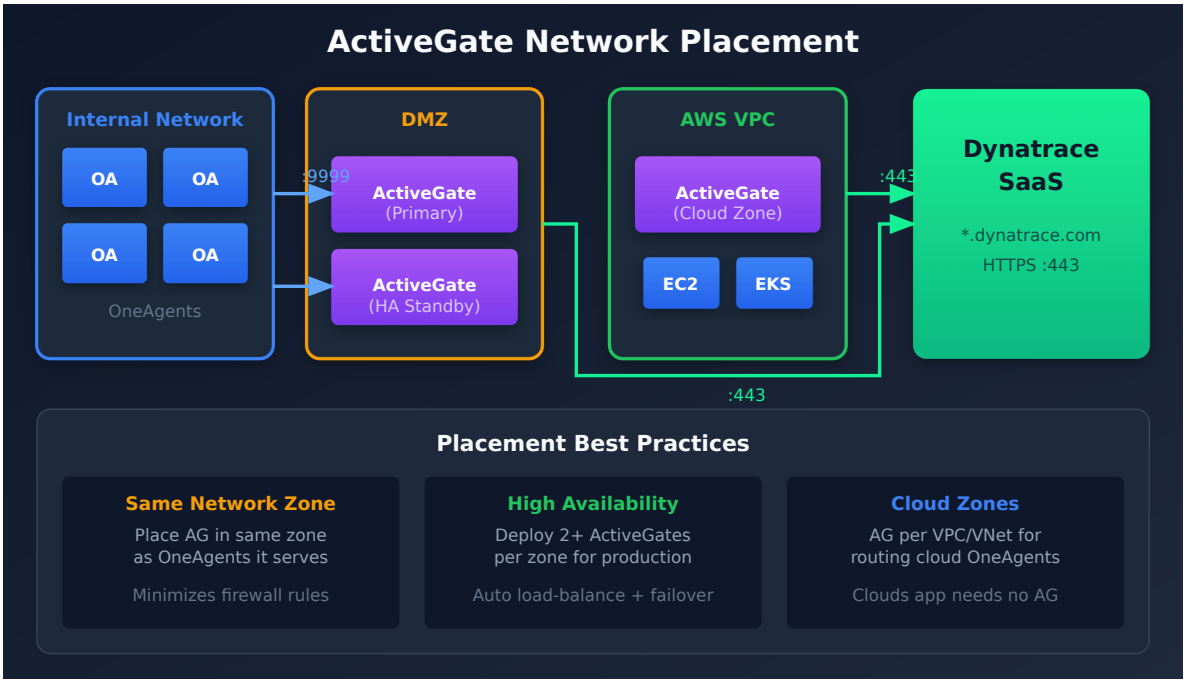
Network Zone	Hosts	ActiveGates	Shape class per AG	vCPU each	RAM each
Production DMZ	200	2	c6i.large (~800 hosts)	2	3.75 GiB
Production Internal	800	2	c6i.xlarge (~1,800 hosts)	4	7.5 GiB
Development	150	1	c6i.large (~800 hosts)	2	3.75 GiB
Total	**1,150**	**5**		**14 vCPU**	**26.25 GiB**

Note why Production Internal is sized at c6i.xlarge for only 800 hosts across two nodes: each node must be able to carry the **whole** zone when its partner is down, so the survivor's capacity — not the average load — sets the shape. FAQ-10 works that survivor-capacity math out properly, including for zones larger than two nodes.

4. Where to Deploy?

Network Zone Strategy

Deploy ActiveGates based on network segmentation:



Placement Rules

Rule	Description
Same network zone	AG should be in same zone as OneAgents it serves
Outbound access	AG needs HTTPS to `*.dynatrace.com`
Inbound from agents	OneAgents connect to AG on port 9999
Low latency	Place AG close to monitored workloads

Cloud-Specific Guidance

Cloud	Recommendation
AWS	Deploy in management VPC or shared services subnet
Azure	Hub VNet with peering to spoke VNets
GCP	Shared VPC host project
Kubernetes	Deploy as StatefulSet or standalone VM outside cluster

5. Generating Tokens

Downloading the ActiveGate installer needs one of two token types, depending on your environment. *"Classic access tokens don't exist in latest environments"*; environments migrating to Latest Dynatrace have both, and SaaS 1.343 says *"We recommend platform tokens over classic API tokens for new integrations."*

Environment	Token	Required scope	Request header
Latest Dynatrace (platform tokens only)	Platform token (**My platform tokens**) or OAuth client	`fleet-management:activegates:download`	`Authorization: Bearer <token>`
Classic or hybrid (classic access tokens still exist)	Access token (**Access Tokens** app → **Generate new token**)	`InstallerDownload` (*PaaS integration - Installer download*)	`Authorization: Api-Token <token>`

No further scopes are needed to install an ActiveGate. Scopes for the APIs you call later (extensions, synthetic locations) belong on the tokens that call them. The **Install ActiveGate** wizard in **Discovery & Coverage** can also generate an installer token for you.

Token Naming Convention

Use descriptive names:
- `prod-activegate-dmz`
- `aws-activegate-useast1`
- `k8s-activegate-cluster1`

> **Sources:** Deployment API - Download latest ActiveGate (DT docs) — *"Platform Token / OAuth: Required scope: fleet-management:activegates:download"*, Upgrade from access tokens classic (DT docs), SaaS 1.343 (DT docs).

6. Installation Methods

Linux Installation

```
# Download the installer (platform token; for a classic access token use
# --header="Authorization: Api-Token {token}" instead)
wget -O Dynatrace-ActiveGate.sh \
  --header="Authorization: Bearer {platform-token}" \
  "https://{tenant-id}.live.dynatrace.com/api/v1/deployment/installer/gateway/unix/latest"

# Make executable and run
chmod +x Dynatrace-ActiveGate.sh
sudo ./Dynatrace-ActiveGate.sh
```

Windows Installation

```
# Download the installer (classic access token: "Api-Token {token}" instead of "Bearer ...")
Invoke-WebRequest -Uri "https://{tenant-
id}.live.dynatrace.com/api/v1/deployment/installer/gateway/windows/latest" `
  -Headers @{ Authorization = "Bearer {platform-token}" } -OutFile Dynatrace-ActiveGate.exe

# Run the installer
.\Dynatrace-ActiveGate.exe
```

> **Deprecation note (SaaS 1.343, July 2026):** the classic ActiveGate deployment API used above (`/api/v1/deployment/installer/gateway/...`) is **deprecated** in favor of the Latest Dynatrace deployment REST API, which supports **platform tokens** with fine-grained OAuth scopes and adds a REST endpoint for public component-image URIs. Its GA/enabled-by-default status arrives with the staged SaaS 1.343 tenant rollout (from mid-July 2026) — verify availability in your tenant. The classic endpoints continue to work during the deprecation period — plan new automation against the platform API and migrate existing scripts on your next maintenance touch.

Container Deployment

Dynatrace documents a containerized ActiveGate only *"using a StatefulSet on Kubernetes/OpenShift"* — see ActiveGate container image (DT docs) and section 7 below. A standalone `docker run` / Podman ActiveGate is not a documented deployment; use the Linux or Windows installer on a VM instead.

Installation Parameters

Parameter	Purpose	Example
`--set-network-zone`	Assign to network zone	`--set-network-zone=dmz`
`--set-group`	Group for management	`--set-group=production`

Synthetic is not an installer flag on a routing ActiveGate: install a separate Synthetic-enabled ActiveGate from the **Install ActiveGate** wizard (**Purpose** → **Synthetic**) — see the note in section 1.

> **Sources:** Deployment API - Download latest ActiveGate (DT docs) — *"GET SaaS https://{your-environment-id}.live.dynatrace.com/api/v1/deployment/installer/gateway/{osType}/latest"*, Customize ActiveGate installation on Linux (DT docs).

7. Kubernetes Deployment (Detailed)

Deploying ActiveGate in Kubernetes requires careful consideration of where, how, and when to use containerized ActiveGates vs. traditional VM deployments.

When to Deploy ActiveGate in Kubernetes

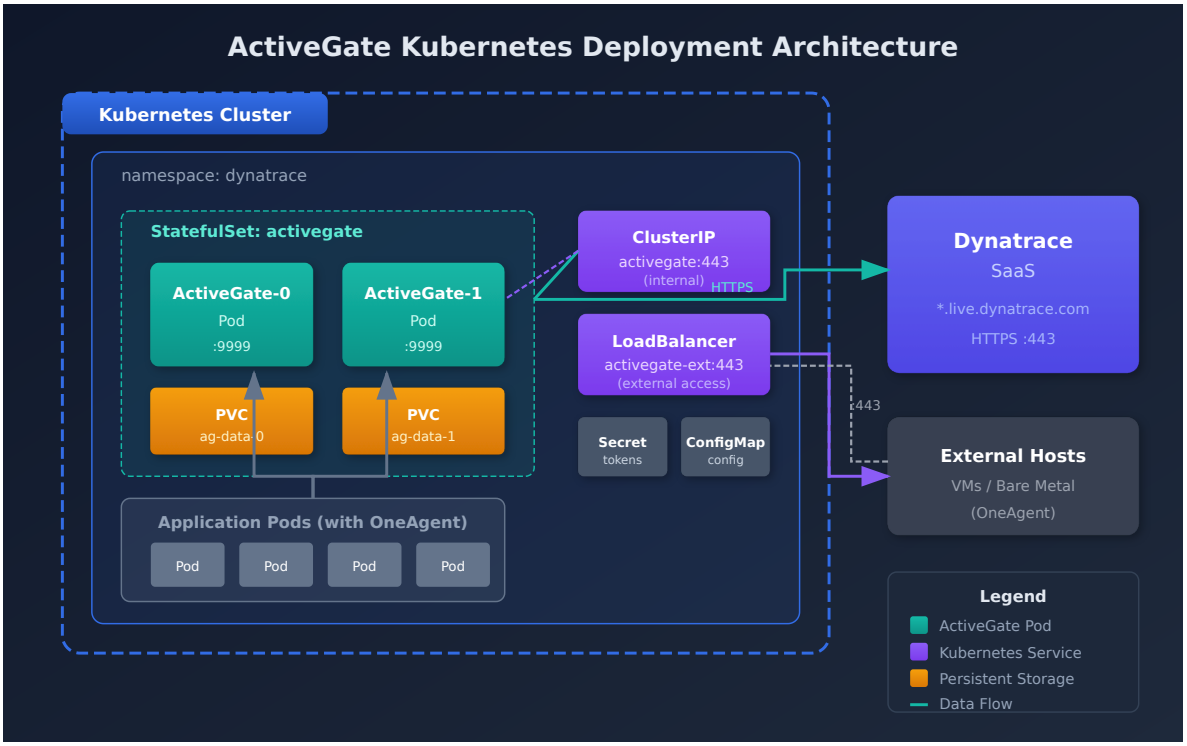
Scenario	Deploy in K8s?	Reasoning
Cluster-only monitoring	✅ Yes	Co-located with workloads, simplifies networking

Scenario	Deploy in K8s?	Reasoning
Routing for in-cluster OneAgents	✅ Yes	Lower latency, no external hops
Extensions 2.0 for K8s resources	✅ Yes	Direct access to cluster APIs
Multi-cluster routing	⚠️ Consider	May need external AG for cross-cluster
Private synthetic monitoring	❌ Usually VM	Synthetic requires stable network, not pod restarts
Routing for external hosts	❌ VM preferred	External hosts need stable, routable IP
Air-gapped environments	❌ VM preferred	Often need dedicated proxy tier

When NOT to Deploy in Kubernetes

Scenario	Why VM is Better
Routing for VMs/bare metal	VMs need stable IPs, not pod IPs
Private synthetic locations	Synthetic tests sensitive to pod restarts
Extensions polling external systems	External systems may not allow K8s egress IPs
Compliance requiring dedicated hosts	Some compliance requires dedicated hardware

Deployment Architecture



Kubernetes ActiveGate Sizing

Dynatrace recommends *"two sets of ActiveGates for production deployments"*: one for Kubernetes platform monitoring (including Prometheus and KSPM), one for OneAgent traffic routing and telemetry ingest. Both are sized by cluster size — pods first, nodes second:

Cluster size	Pods	Nodes
Small	<1,000	Up to 25
Medium	1,000–5,000	Up to 100
Large	5,000–20,000	Up to 500

Kubernetes platform monitoring ActiveGate

Cluster size	CPU request	CPU limit	Memory request	Memory limit
Small	200m	1000m	6 GiB	6 GiB
Medium	1000m	2000m	10 GiB	10 GiB
Large	2000m	4000m	12 GiB	12 GiB

OneAgent traffic routing ActiveGate

Cluster size	CPU request	CPU limit	Memory request	Memory limit	Replicas
Small	250m	1000m	2 GiB	2 GiB	3
Medium	500m	2000m	4 GiB	4 GiB	3
Large	1000m	4000m	6 GiB	6 GiB	6

Platform monitoring needs far more memory than routing — budget for it before you put both jobs on one ActiveGate. The guide's advice on limits: *"Use CPU limits only if required by policy."* OTLP log ingest through the ActiveGate adds replicas. The guide gives a throughput per replica (about 350, 650 and 1,200 MB/min for the small, medium and large configurations) and the formula *"additional replicas = ceil(expected log traffic (MB/min) ÷ throughput per replica)"*.

> **Sources:** Size Dynatrace ActiveGates in Kubernetes (DT docs) — *"One set should cover Kubernetes platform monitoring, including any Prometheus integration and Kubernetes Security Posture Management (KSPM) functionalities."*

Method 1: Dynatrace Operator (Recommended)

The Dynatrace Operator manages ActiveGate lifecycle automatically.

> **Important:** Use `apiVersion: dynatrace.com/v1beta6` for new DynaKubes (`v1beta5` is still served by Operator 1.11.0 but marked deprecated in its CRD from Operator 1.10.0). Operator **1.9.0** removed `v1beta3` from the CRD (*"Applying DynaKube resources using this version will fail"*) and deprecated `v1beta4` ; Operator **1.10.0** (July 15, 2026) stopped serving `v1beta4` (its CRD sets `served: false` , while its notes repeat *"The `v1beta4` version is now deprecated"*); and Operator **1.11.0** (released 10/01/2026) removes it from the CRD — *"Applying DynaKube resources that still use v1beta4 will fail."* Check `kubectl get dynakube -A -o jsonpath='{.items[*].apiVersion}'` before upgrading the Operator, and move any `v1beta4` DynaKube to `v1beta6` first. The example below uses `v1beta6` (validated against the Operator 1.11.0 CRD). v1beta6 adds OTLP exporter configuration.
>
> **Sources:** Operator 1.9.0 release notes (DT docs) — *"Additionally, `v1beta4` is now deprecated."*, Operator 1.10.0 release notes (DT docs), Operator 1.11.0 release notes (DT docs) — *"The v1beta4 version has been removed from the DynaKube CRD."* `served` /

> `deprecated` per version read from the DynaKube CRD in each release's `kubernetes.yaml` (Operator releases (Dynatrace GitHub)), 10/02/2026.

The example follows the sizing guide's two-set layout for Operator versions earlier than 1.11.0 (the version pinned below): one DynaKube for platform monitoring, one for routing, both sized for a small cluster.

```yaml
# dynakube-activegates.yaml - two ActiveGate sets via the Operator (small cluster)
apiVersion: dynatrace.com/v1beta6
kind: DynaKube
metadata:
  name: k8s-monitoring
  namespace: dynatrace
spec:
  apiUrl: https://{tenant-id}.live.dynatrace.com/api
  tokens: dynakube              # secret created below
  networkZone: kubernetes       # network zone for the ActiveGate (and OneAgent) pods

  activeGate:
    capabilities:
      - kubernetes-monitoring   # K8s API monitoring
    resources:
      requests:
        cpu: "200m"
        memory: "6Gi"
      limits:
        memory: "6Gi"           # add a CPU limit (1000m) only if policy requires one

    # Tolerations and node selector for dedicated monitoring nodes (optional)
    tolerations:
      - key: "dedicated"
        operator: "Equal"
        value: "dynatrace"
        effect: "NoSchedule"
    nodeSelector:
      node-type: monitoring
---
apiVersion: dynatrace.com/v1beta6
kind: DynaKube
metadata:
  name: agents
  namespace: dynatrace
spec:
  apiUrl: https://{tenant-id}.live.dynatrace.com/api
  tokens: dynakube
  networkZone: kubernetes
  # add your oneAgent: section here (see ONBRD-05)

  activeGate:
    capabilities:
      - routing                 # route OneAgent traffic
    resources:
      requests:
        cpu: "250m"
        memory: "2Gi"
      limits:
        memory: "2Gi"
    replicas: 3

    # Labels
    labels:
      app.kubernetes.io/component: activegate
```

> **Operator 1.11.0+:** the sizing guide shows a single DynaKube instead, with platform monitoring under `spec.kubernetesMonitoring` — *".spec.kubernetesMonitoring is mutually exclusive with the kubernetes-monitoring capability in .spec.activeGate"*. The `dynatrace-api` capability is left out here: the DynaKube

> reference marks it *"A custom certificate is required for this capability"* (`tlsSecretName`). `spec.networkZone` *"Sets a network zone for the OneAgent and ActiveGate Pods."*
>
> **Sources:** DynaKube parameters (DT docs), Size Dynatrace ActiveGates in Kubernetes (DT docs).

Install with Operator:

```
# Create namespace
kubectl create namespace dynatrace

# Create secret with tokens. On Latest Dynatrace, the Operator docs ("Tokens and permissions")
# direct you to two PLATFORM tokens (Operator token + Data Ingest token) on a dedicated service user;
# existing classic access tokens continue to be accepted. Classic path shown here:
# apiToken = the Operator token. Create it in Access Tokens from the "Kubernetes: Dynatrace
# Operator" template: an ActiveGate DynaKube needs activeGateTokenManagement.create on top of
# InstallerDownload (see ONBRD-05 section 5) - an installer-only token is not enough.
# the separate paasToken secret field is deprecated as of Operator 1.10.0 (still accepted).
# From Operator 1.10.0 a Platform Token is also accepted in place of the classic access token.
kubectl -n dynatrace create secret generic dynakube \
  --from-literal=apiToken=&lt;your-api-token&gt;

# Install Dynatrace Operator from the OCI registry, pinned to the version you validated
helm upgrade dynatrace-operator oci://public.ecr.aws/dynatrace/dynatrace-operator \
  --version 1.10.2 \
  --namespace dynatrace \
  --install --atomic \
  --set installCRD=true

# Apply DynaKube configuration
kubectl apply -f dynakube-activegates.yaml
```

> **Install from the OCI registry.** The Operator 1.11.0 release notes direct Helm installs to `oci://public.ecr.aws/dynatrace/dynatrace-operator` : the legacy `dynatrace/helm-charts` repository is archived, and *"If you can't use OCI, point your Helm repository to dynatrace/dynatrace-operator instead"* (`helm repo add dynatrace https://raw.githubusercontent.com/Dynatrace/dynatrace-operator/main/config/helm/repos/stable`). `1.10.2` is the version validated across these notebooks; Operator **1.11.0** (released 10/01/2026) is the newest — validate it on a non-production cluster before moving the pin.
>
> **Sources:** Operator 1.11.0 release notes (DT docs) — *"The Helm repository located in dynatrace/helm-charts is archived and no longer receives updates."*

Method 2: Hand-Managed StatefulSet (Documented Container Image)

The Operator (Method 1) is the only Helm-based path: the Dynatrace Helm repository publishes the `dynatrace-operator` chart, not a standalone ActiveGate chart.

If you must run ActiveGate without the Operator, follow ActiveGate container image (DT docs) exactly rather than adapting a generic manifest. The documented StatefulSet takes `DT_TENANT` , `DT_SERVER` , `DT_ID_SEED_NAMESPACE` , `DT_ID_SEED_K8S_CLUSTER_ID` , `DT_CAPABILITIES` and `DT_DEPLOYMENT_METADATA` , mounts an **authentication token** at `/var/lib/dynatrace/secrets/tokens` , pulls the image from a documented registry, and uses `/rest/state` (liveness) and `/rest/health` (readiness) probes. There is no `DT_API_TOKEN` variable, and none of this is needed on the Operator path.

The Service, LoadBalancer, PodDisruptionBudget and anti-affinity snippets below are generic Kubernetes — adjust their `selector` labels to match the labels on your ActiveGate pods.

Exposing ActiveGate for External Access

If external hosts need to route through the K8s ActiveGate:

```
# Option 1: LoadBalancer Service
apiVersion: v1
kind: Service
metadata:
  name: activegate-external
  namespace: dynatrace
  annotations:
    # AWS NLB
    service.beta.kubernetes.io/aws-load-balancer-type: "nlb"
    service.beta.kubernetes.io/aws-load-balancer-scheme: "internal"
spec:
  type: LoadBalancer
  selector:
    app: activegate
  ports:
    - port: 443
      targetPort: 9999
---
# Option 2: Ingress (with TLS passthrough)
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: activegate-ingress
  namespace: dynatrace
  annotations:
    nginx.ingress.kubernetes.io/ssl-passthrough: "true"
    nginx.ingress.kubernetes.io/backend-protocol: "HTTPS"
spec:
  ingressClassName: nginx
  rules:
    - host: activegate.internal.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: activegate
                port:
                  number: 443
```

Platform-Specific Configurations

Amazon EKS

```
# EKS-specific annotations
apiVersion: v1
kind: Service
metadata:
  name: activegate
  namespace: dynatrace
  annotations:
    service.beta.kubernetes.io/aws-load-balancer-type: "nlb"
    service.beta.kubernetes.io/aws-load-balancer-internal: "true"
    service.beta.kubernetes.io/aws-load-balancer-cross-zone-load-balancing-enabled: "true"
spec:
  type: LoadBalancer
  # ...
```

Azure AKS

```
# AKS-specific annotations
apiVersion: v1
kind: Service
metadata:
  name: activegate
  namespace: dynatrace
  annotations:
    service.beta.kubernetes.io/azure-load-balancer-internal: "true"
spec:
  type: LoadBalancer
  # ...
```

Google GKE

```
# GKE-specific annotations
apiVersion: v1
kind: Service
metadata:
  name: activegate
  namespace: dynatrace
  annotations:
    cloud.google.com/load-balancer-type: "Internal"
spec:
  type: LoadBalancer
  # ...
```

High Availability in Kubernetes

Configuration	Setting
Replicas	Routing ActiveGate: 3 (small/medium cluster), 6 (large), per the sizing guide
Pod Anti-Affinity	Spread across nodes
PodDisruptionBudget	minAvailable: 1

Configuration	Setting
Resource Requests	Guarantee scheduling

```
# Pod Anti-Affinity for HA
spec:
  affinity:
    podAntiAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        - labelSelector:
            matchLabels:
              app: activegate
          topologyKey: kubernetes.io/hostname
---
# PodDisruptionBudget
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: activegate-pdb
  namespace: dynatrace
spec:
  minAvailable: 1
  selector:
    matchLabels:
      app: activegate
```

8. Verifying Deployment

After installation, verify ActiveGate is connected and healthy. ActiveGates surface in Grail as the `ACTIVEGATE` **Smartscape node**, so the queries below work directly in a notebook — no REST call needed.

> **A note on the classic path (ActiveGate 1.343, July 2026):** ActiveGate 1.343 deprecates the classic `GET /api/v2/activeGates` endpoints in favor of this Smartscape node. Separately, ActiveGates have **never** been reachable through DQL `fetch` — there is no `dt.entity.active_gate` entity type in any spelling. Worth knowing precisely how that fails: `fetch dt.entity.active_gate` does not error — it returns **zero rows**, which is indistinguishable from "this environment has no ActiveGates." Treat an empty result from a classic entity fetch as a signal to check the entity type exists at all. `smartscapeNodes "ACTIVEGATE"` is the DQL path. The classic **Entities API v2** selector (`GET /api/v2/entities?entitySelector=type("ENVIRONMENT_ACTIVE_GATE")`) is a different surface from DQL and may still respond during the deprecation period — use it if you need a REST fallback for a tenant that has not yet received 1.343, and migrate the automation on your next maintenance touch.

```
// ActiveGate inventory - every ActiveGate reporting to this environment,
// with its group, network zone, enabled modules, and host OS.
// Live-verified 07/30/2026. Costs nothing: Smartscape nodes scan 0 bytes of Grail.
smartscapeNodes "ACTIVEGATE"
| fields name, dt.active_gate.id, dt.active_gate.version, dt.active_gate.group.name,
         dt.network_zone.id, is_containerized, os.type, os.version, modules
| sort name asc
```

```
// ActiveGate version spread - find the laggards before an update window.
// More than one row means the fleet is not uniform; sort ascending puts the
// oldest version first. Cross-check the OS end-of-support table in section 3
// against the os.version values from the inventory query above.
// ids is listed because names can repeat: two ActiveGate pods in different
// clusters can share a pod name (e.g. dynakube-activegate-0).
smartscapeNodes "ACTIVEGATE"
| summarize {activegates = count(), names = collectDistinct(name), ids =
collectDistinct(dt.active_gate.id)},
    by:{dt.active_gate.version}
| sort dt.active_gate.version asc
```

```
// ActiveGate count per group and network zone - the HA check.
// Any row showing 1 is a single point of failure for that zone; production
// zones should show 2 or more (see High Availability in section 3).
smartscapeNodes "ACTIVEGATE"
| summarize activegates = count(), by:{dt.active_gate.group.name, dt.network_zone.id}
| sort activegates desc
```

Verification via UI

Location: Fleet Management → ActiveGates (SaaS 1.343+: *"You can now monitor and manage your ActiveGates in the new Fleet Management."*); on older tenants, **Deployment status** → **ActiveGates**

Check for:
- ✅ Status: Connected
- ✅ Version: Latest or recent
- ✅ Modules: Expected capabilities enabled

Verification Commands

Linux:

```
# Check service status
sudo systemctl status dynatracegateway

# Check connectivity
curl -k https://localhost:9999/rest/health   # expect: RUNNING

# View logs
sudo tail -100 /var/log/dynatrace/gateway/gateway.log
```

Windows:

```
# Check service status
Get-Service -Name "Dynatrace Gateway"

# Check connectivity
Invoke-WebRequest -Uri "https://localhost:9999/rest/health" -SkipCertificateCheck
```

> **Sources:** ActiveGate default installation settings for Windows (DT docs) — *"ActiveGate Dynatrace Gateway The main ActiveGate service."*, SaaS 1.343 (DT docs).

9. Troubleshooting

Common Issues

Issue	Cause	Solution
Not appearing in UI	Network blocked	Check firewall for 443 outbound
Shows disconnected	Service stopped	Restart dynatracegateway service
OneAgents not routing	Wrong network zone	Verify zone configuration
High memory usage	Too many agents	Add another AG or increase RAM
Certificate errors	Self-signed cert issue	Trust Dynatrace CA or use custom cert

Network Verification

```
# Test outbound to Dynatrace
curl -v https://{tenant-id}.live.dynatrace.com/rest/health   # expect: RUNNING

# Test ActiveGate is listening
netstat -tlnp | grep 9999

# Test from OneAgent host
curl -k https://{activegate-ip}:9999/rest/health
```

Log Locations

OS	Log Path
Linux	`/var/log/dynatrace/gateway/`
Windows	`C:\ProgramData\dynatrace\gateway\log\`

10. Next Steps

With ActiveGate deployed:

1. **ONBRD-04: Cloud & SaaS Integrations** — Connect AWS / Azure (Clouds app, or classic polling through your AG) and GCP (no AG on SaaS), and assign remote extensions to an AG group
2. **ONBRD-05: Deploying OneAgent** — Now deploy agents that route through ActiveGate
3. Configure network zones if using multiple ActiveGates
4. Enable additional modules (synthetic, extensions) as needed
5. Set up monitoring for ActiveGate itself

Where to Go Deeper

- **K8S series** (15 notebooks) — Cluster monitoring, DynaKube depth, GitOps for K8s deployments
- **CLOUD series** (9 notebooks) — Per-cloud integration deep dives (AWS, Azure, GCP)
- **AUTOM series** — Configuration automation for ActiveGate deployment at scale

Deployment Checklist

Traditional (VM/Bare Metal)

- [ ] ActiveGate deployed in each required network zone
- [ ] At least 2 per zone for HA (production)
- [ ] Status shows "Connected" in Dynatrace
- [ ] Firewall rules configured (443 out, 9999 in)
- [ ] Network zones configured (if applicable)
- [ ] Sizing appropriate for expected OneAgent count

Kubernetes

- [ ] Dynatrace Operator installed (recommended)
- [ ] DynaKube CR applied with activeGate configuration
- [ ] Separate platform-monitoring and routing ActiveGates for production
- [ ] Replicas and resource requests/limits set per the K8s ActiveGate sizing guide
- [ ] PodDisruptionBudget created
- [ ] Pod anti-affinity configured for spread across nodes
- [ ] Service type appropriate (ClusterIP vs LoadBalancer)

Summary

In this notebook, you learned:

- What ActiveGate does and its capabilities
- When ActiveGate is required vs. optional
- Hardware requirements, the published per-shape capacity figures, and why FAQ-10 owns the sizing decision
- Which operating systems are currently supported, and which ones lose support inside the next six months
- Where to place ActiveGates in your network
- Installation methods for Linux and Windows, and why a containerized ActiveGate means a Kubernetes/OpenShift StatefulSet
- **Kubernetes deployment** using the Operator, or the documented hand-managed StatefulSet
- Platform-specific configurations for EKS, AKS, and GKE
- How to verify successful deployment with `smartscapeNodes "ACTIVEGATE"` inventory, version, and HA queries

---

References

General

- ActiveGate Overview
- ActiveGate Installation
- Network Zones

- Linux ActiveGate hardware and system requirements
- Windows ActiveGate hardware and system requirements
- End-of-support announcements
- Private Synthetic Locations
- Clouds App

Kubernetes

- Dynatrace Operator
- Dynatrace Operator GitHub
- DynaKube parameters — ActiveGate configuration (DT docs)
- DynaKube parameters (DT docs)
- ActiveGate container image (DT docs)
- Tokens and permissions (DT docs)
- Size Dynatrace ActiveGates in Kubernetes (DT docs)
- Helm Chart Repository

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.